

7	Accounts to create	8	Software to install	14	Python packages	29	Checklist items
---	--------------------	---	---------------------	----	-----------------	----	-----------------

SECTION 1 — Online Accounts to Create (All Free, No Credit Card)

Create these accounts before writing a single line of code. All are free and require only an email address except NVIDIA NIM which needs phone verification.

Platform	Why you need it	URL	When
GitHub (Main account)	Your primary developer — high DTS 'Priya'. Used to host the demo repo, raise clean PRs, post inline review comments via API.	github.com/signup	Day 1 — first thing
GitHub (Second account)	Low DTS developer 'Rahul'. Raises the buggy PR during demo. Judges see the contrast between DTS-92 and DTS-15 review depth.	github.com/signup	Day 1
NVIDIA NIM / NGC	Free API key for all AI models — Qwen3, Codex, Devstral-2, Llama 4 Maverick, NV-EmbedQA. Needs phone number OTP.	build.nvidia.com	Day 1 — critical
Discord	Replaces Microsoft Teams for alerts. Create a server, add #devguardian-alerts channel, generate a webhook URL. Free forever.	discord.com/register	Day 2
ngrok	Tunnels localhost:8000 to a public HTTPS URL so GitHub webhook can reach your FastAPI server during demo.	ngrok.com/signup	Day 4
Railway (optional)	Free-tier cloud hosting for FastAPI. Only needed if you want a permanent URL instead of ngrok for the demo.	railway.app	Day 5 — optional
GitHub Codespaces (optional)	Code from browser if laptop is slow. 60 free hours/month. Comes with GitHub account automatically.	github.com/codespaces	Optional

SECTION 2 — Software to Install on Your Laptop

Software	Why needed	Download from	When
Python 3.11+	Core language. MUST check 'Add to PATH' during Windows install.	python.org/downloads	Day 1 — install first
VS Code	Code editor. Lightweight, fast, best Python support.	code.visualstudio.com	Day 1
Git	Version control. Required to push branches and simulate PR workflow.	git-scm.com/downloads	Day 1
ngrok CLI	Expose localhost:8000 publicly. Extract binary to any folder in PATH.	ngrok.com/download	Day 4

Postman	Test FastAPI endpoints and webhook payloads manually.	postman.com/downloads	Day 2
DB Browser for SQLite	Visual inspector for devguardian.db — see DTS scores live.	sqlitebrowser.org	Day 2
GitHub Desktop	GUI for branch management. Optional if team prefers CLI.	desktop.github.com	Optional
Semgrep CLI	Static analysis engine. Installed via pip — no separate download.	<code>pip install semgrep</code>	Day 3 — via pip

VS Code Extensions to install (all free — search by ID in Extensions panel)

Name	Extension ID	What it does
Python	<code>ms-python.python</code>	IntelliSense, debugging, linting
Pylance	<code>ms-python.vscode-pylance</code>	Fast type checking and autocomplete
SQLite Viewer	<code>qwtel.sqlite-viewer</code>	Browse devguardian.db inside VS Code
REST Client	<code>humao.rest-client</code>	Test API endpoints without leaving editor
Thunder Client	<code>rangav.vscode-thunder-client</code>	Postman-style testing inside VS Code
GitLens	<code>eamodio.gitlens</code>	Git blame and commit history inline
GitHub Copilot	<code>github.copilot</code>	AI code assist — free for students

SECTION 3 — Python Packages (requirements.txt)

Create requirements.txt in your project root, paste all lines below, then run one command:

```
pip install -r requirements.txt
```

Package	Version	What it does in DevGuardian
<code>fastapi</code>	latest	Web framework — handles webhooks, dashboard routes, API endpoints
<code>uvicorn[standard]</code>	latest	ASGI server that runs FastAPI. Command: <code>uvicorn main:app --reload</code>
<code>requests</code>	latest	HTTP calls to NVIDIA NIM API, GitHub API, Discord webhooks
<code>httpx</code>	latest	Async HTTP client for non-blocking NVIDIA NIM calls inside FastAPI
<code>chromadb</code>	latest	Local vector database for Module 2 (DNA Fingerprint Engine)
<code>semgrep</code>	latest	Static analysis — 500+ security rules for Module 3
<code>python-dotenv</code>	latest	Loads .env API keys — never hardcode keys in code
<code>jinja2</code>	latest	HTML templates for FastAPI DTS dashboard
<code>pandas</code>	latest	Data analysis for DTS calculation — aggregating PR history
<code>pytest</code>	latest	Unit test framework for all 6 modules
<code>pytest-asyncio</code>	latest	Async support for testing FastAPI endpoints with pytest
<code>python-multipart</code>	latest	Required by FastAPI to handle form data in webhook payloads
<code>aiofiles</code>	latest	Async file operations inside FastAPI routes

openai	latest	OpenAI SDK — used as NVIDIA NIM client (same API format, different base_url)
--------	--------	--

SECTION 4 — API Keys and .env File

.env variable	Example value	How to get it	When
NVIDIA_API_KEY	nvapi-xxxxx	build.nvidia.com/settings/api-keys → Generate Key → copy immediately (shown once only)	Day 1
GITHUB_TOKEN	ghp_xxxxx	github.com → Settings → Developer Settings → Personal Access Tokens (classic) → Scopes: repo, read:user, write:discussion	Day 1
GITHUB_TOKEN_RAHUL	ghp_yyyyy	Same as above but from the second (Rahul) GitHub account. Used to submit PR as low-DTS developer.	Day 1
GITHUB_WEBHOOK_SECRET	any string	Make up any random string e.g. devguardian2024. Enter same value in GitHub webhook settings AND this .env file.	Day 1
GITHUB_REPO_OWNER	your-username	Your primary GitHub username. Used in all GitHub API calls.	Day 1
GITHUB_REPO_NAME	devguardian-demo	The name of the test repo you create on GitHub.	Day 1
TEAMS_WEBHOOK_URL	discord URL	Discord server → channel settings → Integrations → Create Webhook → Copy URL. Looks like: discord.com/api/webhooks/...	Day 2

.env file template — copy exactly into your project root

```
NVIDIA_API_KEY=nvapi-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
GITHUB_TOKEN=ghp_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
GITHUB_TOKEN_RAHUL=ghp_yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy GITHUB_WEBHOOK_SECRET=devguardian2024
GITHUB_REPO_OWNER=your-github-username GITHUB_REPO_NAME=devguardian-demo
TEAMS_WEBHOOK_URL=https://discord.com/api/webhooks/xxx/yyy DATABASE_PATH=devguardian.db PORT=8000
```

SECTION 5 — NVIDIA NIM Free Models for DevGuardian (2026)

All models below are verified working as of June 2026 on NVIDIA NIM. Note: Codestral is currently showing 404 — use Qwen3 Coder 480B instead for all code tasks. Rate limit: ~40 requests/minute shared across all models.

Model ID (use exactly as written)	Used in DevGuardian module	Notes
qwen/qwen3-coder-480b-instruct	Code review (Deep), test generation, security explanation	480B MoE, 256K context. Best code model on platform. Replaces Codestral.
mistralai/devstral-2-123b	Code review (Standard), PR analysis	71% accuracy on code benchmarks. Best overall accuracy for diverse codebases.
meta/llama-4-maverick	Code review (Shallow), team reports, deploy gate	Most popular model — 22M uses. Multimodal. Fast, reliable, great language quality.
nvidia/nemotron-super-49b-instruct	Fallback model, cross-service reasoning	NVIDIA's own model. 262K context. Best on hard multi-file analysis tasks.

nvidia/nv-embedqa-e5-v5	DNA Fingerprint Engine — embed code into ChromaDB	Embedding model. Converts code to vectors for semantic similarity search.
---	---	---

Model routing — which model for which DTS depth

Condition	Action	Model	Expected time
DTS 80–100	Shallow review	meta/llama-4-maverick	~8 sec, quick scan only
DTS 50–79	Standard review	mistralai/devstral-2-123b	~25 sec, full review
DTS 20–49	Deep review	qwen/qwen3-coder-480b-instruct	~45 sec, full audit + tests
DTS 0–19	Block PR	No AI call — block immediately	Notify team lead via Discord
Module 2	DNA Fingerprint	nvidia/nv-embedqa-e5-v5	Embed past PRs → ChromaDB
Module 5/6	Reports + Gate	meta/llama-4-maverick	Summaries and risk reports

SECTION 6 — Project Folder Structure

```
devguardian/ +-- main.py # FastAPI app + GitHub webhook handler (entry point) +-- dts_engine.py #
Module 1 – DTS formula, SQLite read/write, depth routing +-- dna_engine.py # Module 2 – ChromaDB
embed + style violation query +-- security_scanner.py # Module 3 – Semgrep subprocess + NIM
explanation +-- test_generator.py # Module 4 – AST parse untested functions + Qwen3 Coder +--
risk_reporter.py # Module 5 – weekly team report via Llama 4 + Discord +-- deploy_gate.py #
Module 6 – deployment Red/Yellow/Green gate +-- github_adapter.py # All GitHub API calls (PR
diff, post comments, get issues) +-- nim_client.py # NVIDIA NIM calls with fallback chain on 429
+-- notifier.py # POST to Discord webhook (Teams-compatible format) +-- seeder.py # Seed 5 demo
developers into SQLite for demo prep +-- templates/ | +-- dashboard.html # DTS dashboard UI
served at localhost:8000/dashboard +-- tests/ | +-- test_dts.py # Unit tests for DTS formula and
routing logic | +-- test_security.py # Unit tests for Semgrep integration | +-- test_webhook.py #
Integration tests for GitHub webhook handler +-- .env # API keys – add to .gitignore immediately
+-- .gitignore # Must include: .env, *.db, __pycache__, .venv +-- requirements.txt # All 14
packages +-- devguardian.db # SQLite database (auto-created on first run) +-- README.md # Project
description for GitHub
```

SECTION 7 — 5-Day Hackathon Build Flow

Follow this exact order. Each day builds on the previous. Do not skip ahead — each step validates the one before it.

DAY 1 Foundation — accounts, repo, skeleton

1	Create both GitHub accounts (Priya + Rahul). Generate PATs for both.
2	Create NVIDIA NIM account at build.nvidia.com. Get API key starting with nvapi-.
3	Install Python 3.11, VS Code, Git. Verify: python --version, git --version.
4	Create GitHub repo 'devguardian-demo'. Add buggy Python file (SQL injection + hardcoded secret).
5	Create project folder structure. Create .env file with all variables.
6	Run: pip install -r requirements.txt. Verify all packages install without errors.
7	Run test_nim.py — confirm NVIDIA NIM API responds OK for Llama 4 Maverick.

DAY 2 Core intelligence — DTS engine + database

1	Run seeder.py — creates devguardian.db with 5 developers and correct DTS scores.
2	Open devguardian.db in DB Browser for SQLite — confirm all 5 rows visible with correct scores.
3	Build dts_engine.py — implement calculate_dts() formula and get_review_depth() routing.
4	Unit test: call calculate_dts for all 5 seeded developers. Confirm Shallow/Standard/Deep/Block routing.
5	Create Discord server. Add #devguardian-alerts channel. Generate webhook URL. Save to .env.
6	Test Discord webhook: POST a test JSON message. Confirm it appears in Discord channel.
7	Build notifier.py — wraps Discord POST. Test: python -c 'from notifier import send_alert; send_alert("Test")'

DAY 3 AI review + security + DNA fingerprint

- 1 Build `nim_client.py` — Qwen3 Coder for deep, Devstral for standard, Llama 4 for shallow. Include 429 fallback.
- 2 Test `nim_client.py` with a sample PR diff — confirm structured JSON review response.
- 3 Build `security_scanner.py` — run Semgrep as subprocess on test file, parse JSON, send to NIM for explanation.
- 4 Test: `semgrep --config=auto app/database.py --json` — confirm SQL injection and hardcoded secret findings.
- 5 Build `dna_engine.py` — embed 20+ Python files from any open-source repo into ChromaDB.
- 6 Test DNA: submit a differently-styled snippet, confirm style violations detected with similarity scores.

DAY 4 Test generator + webhook + ngrok tunnel

- 1 Build `test_generator.py` — Python AST finds untested functions, Qwen3 Coder generates pytest tests.
- 2 Test: run `test_generator` on a Python file with 3 functions and no test file. Confirm tests generated.
- 3 Build `main.py` FastAPI app — `/webhook` endpoint, `/dashboard` endpoint, `/health` check.
- 4 Run: `uvicorn main:app --reload`. Open browser at `localhost:8000/dashboard` — confirm DTS table visible.
- 5 Install ngrok. Run: `ngrok config add-authtoken YOUR_TOKEN`. Then: `ngrok http 8000`.
- 6 Copy ngrok URL (e.g. `https://abc123.ngrok.io`). Add to GitHub webhook settings for your demo repo.
- 7 Test webhook: open a PR from Rahul account. Confirm FastAPI receives the event (check terminal logs).

DAY 5 Full integration + demo prep + dry run

- 1 Build `risk_reporter.py` — aggregate weekly SQLite data, generate anonymised team report via Llama 4.
- 2 Build `deploy_gate.py` — compute average DTS of all authors in release, output Green/Yellow/Red.
- 3 Full end-to-end test: PR from Rahul → deep review fires → inline comments on buggy lines → Discord alert.
- 4 Full end-to-end test: PR from Priya → shallow review fires → clean result in under 10 seconds.
- 5 Pre-create 2 branches: `feature/rahul-auth-fix` (with bugs) and `feature/priya-cleanup` (clean code).
- 6 Do complete 4-step demo dry run 3 times. Time it — target under 2 minutes total.
- 7 Record a backup demo video. Export at 1080p. Save to a USB stick as insurance against wifi failure.

SECTION 8 — Master Setup Checklist (tick off as you go)

#	Task	When	Tool / URL
1	Create GitHub main account (Priya)	Day 1	github.com/signup
2	Create GitHub second account (Rahul)	Day 1	github.com/signup
3	Create NVIDIA NIM account	Day 1	build.nvidia.com
4	Get NVIDIA API key — starts with <code>nvapi-</code>	Day 1	build.nvidia.com/settings/api-keys
5	Install Python 3.11+ with PATH checked	Day 1	python.org/downloads

6	Install VS Code + Python + Pylance extensions	Day 1	code.visualstudio.com
7	Install Git	Day 1	git-scm.com
8	Create GitHub repo devguardian-demo with buggy Python file	Day 1	github.com/new
9	Generate GitHub PAT for Priya account	Day 1	github.com/settings/tokens
10	Generate GitHub PAT for Rahul account	Day 1	github.com/settings/tokens
11	Create full project folder structure (10 Python files)	Day 1	VS Code
12	Create requirements.txt — pip install -r requirements.txt	Day 1	terminal
13	Create .env file with all 7 variables filled in	Day 1	VS Code
14	Run seeder.py — confirm 5 developers in SQLite	Day 2	<code>python seeder.py</code>
15	Test NVIDIA NIM API call — confirm response OK	Day 2	<code>python test_nim.py</code>
16	Build Module 1 DTS Engine — test all 4 depth levels	Day 2	<code>dts_engine.py</code>
17	Create Discord server + #devguardian-alerts + webhook URL	Day 2	discord.com
18	Test Discord webhook — POST a test message	Day 2	<code>python notifier.py</code>
19	Build Module 3 Security Scanner — test on buggy file	Day 3	<code>security_scanner.py</code>
20	Build DNA Fingerprint Engine — embed 20 files to ChromaDB	Day 3	<code>dna_engine.py</code>
21	Build Module 4 Test Generator — Qwen3 generates tests	Day 4	<code>test_generator.py</code>
22	Install ngrok + configure auth token	Day 4	ngrok.com/download
23	Run: ngrok http 8000 — get public URL	Day 4	terminal
24	Add ngrok URL to GitHub webhook settings	Day 4	github.com repo settings
25	E2E test: Rahul PR → deep review → inline comments → Discord alert	Day 5	full flow
26	E2E test: Priya PR → shallow review in under 10 sec	Day 5	full flow
27	Build DTS dashboard — open localhost:8000/dashboard	Day 5	<code>main.py</code> + <code>dashboard.html</code>

28	Demo dry run x3 — all 4 steps, time it under 2 min	Day 5	practice
29	Record backup demo video at 1080p	Day 5	screen recorder

DevGuardian — Microsoft Hackathon Theme 06. All costs = Rs 0. All tools free. Total build time = 5 days. Upload this PDF to Claude to restore full project context instantly.